Message to Sender:

The sender operator/ technician must log in to the bank's secure server environment where the core-banking ledger and the bank's financial-messaging engine are installed and running.

From within this bank's secure server environment, the operator prepares or loads the ISO 20022 pacs.008 or MT103 payment message into the bank's financial-messaging engine outbound module, which then handles validation, ledger posting, secure transmission to the receiver, and ACK/NACK monitoring.

The bank's financial messaging engine (e.g., SWIFT interface, ISO 20022 gateway, host-to-host connector, or raw TLS payment transport module) is directly linked to the sender's ledger and is the component responsible for generating, signing, executing the debit, and producing the outbound payment message MT103 or ISO 20022 messages, and delivering it to the receiver.

This bank's messaging engine will establish a direct server-to-server (S2S) connection to the receiver using a raw TLS-encrypted TCP channel (direct IP, non-HTTP).

This transport method—sometimes referred to as a raw-TLS socket, direct IP tunnel, or non-HTTP TLS pipe—provides an authenticated, encrypted delivery path without relying on HTTP, REST, or API endpoints.

Once the secure TLS handshake completes, the messaging engine transmits the framed payment payload over the raw TLS stream, handles message length-prefixing, enforces idempotency and replay protection, and waits for the receiver's ACK/NACK or operational response.

The operator monitors the transmission logs, confirmation states, and any returned status messages until the payment is fully acknowledged.

Key points

✔ The sender logs in to the bank's server/network
✔ The sender triggers the payment manually (realistic for operators)
✔ The payment still flows through the **bank's internal messaging engine**, not by manually crafting TCP packets
✔ The message (MT103 or ISO 20022) is validated, debited, and transmitted through proper host-to-host infrastructure
✔ Receiver responses (ACK/NACK/status) are monitored on that server

Receiver Details:

Server Name:	DEV-CORE-PAY-GW-01
Server IP:	172.67.157.88
Port:	443
Hostname (SNI):	devmindgroup.com
Certificate SHA-256 fingerprint (leaf cert) For OpenSSL-style pinning (CLI, config files)	59:77:D1:92:7D:A4:C5:CC:58:67:DC:6B:44:C2:22:B9:FA:6A:D7:61:5D:DD:FE:40:BA:D3:41:70:69:A0:F7:5D
Certificate SHA-256 fingerprint (leaf cert) For Programmatic pinning (Java, Go, Python, Node, banking engines)	5977d1927da4c5cc5867dc6b44c222b9fa6ad7615dddfe40bad3417069a0f75d
Public Key SHA-256 fingerprint (SPKI pin):	68b3be528835e38b461c73b983dded53e089f22fdcb59cb3d9d730d17c612ae9
Internal IP Range:	172.16.0.0/24, 10.26.0.0/16
DNS Range:	192.168.1.100/24
Bank Name:	DFCU BANK LIMITED
Bank Address:	PLOT 26 KYADONDO ROAD, DFCU TOWERS, KAMPALA, UGANDA
SWIFT Code:	DFCUUGKA
Account Name:	SHAMRAYAN ENTERPRISES
Account Number:	02650010158937

Technical Summary:

1) Required (minimum) to establish a raw TLS S2S connection

- **Receiver IP** — `172.67.157.88`
  (You must be able to reach the host by IP to bypass any DNS/proxies when you want raw S2S.)
- **Receiver Port** — `443`
  (The TCP port the server is listening on.)
- **Receiver Hostname / SNI** — `devmindgroup.com`
  (Most TLS servers present the correct certificate only when the client sends this SNI during the TLS ClientHello.)
- **Way to verify the server certificate** — either:
  - server **SHA256 fingerprint** (we provided one), or
  - a trusted CA chain (full chain PEM) to validate the certificate.

These four are the **practical minimum** for making a trustworthy raw-TLS connection.

Depends on the Type of Sender's Financial Messaging Engine/ Command Line tools or Programming Language, there are several ways for sender operator to establish a **raw TLS-encrypted TCP socket** with **SNI = devmindgroup.com** to the receiver at **172.67.157.88:443**

For Example:

1. openssl s_client -connect 172.67.157.88:443 -servername devmindgroup.com
2. Python TLS Client With SNI Save as: **tls_client.py**

```
import socket
import ssl

SERVER_IP = "172.67.157.88"
PORT = 443
SNI_HOSTNAME = "devmindgroup.com"

# Create raw TCP socket
sock = socket.create_connection((SERVER_IP, PORT))

# Wrap with TLS, including SNI
context = ssl.create_default_context()

tls = context.wrap_socket(sock, server_hostname=SNI_HOSTNAME)
```

Note: This is not a full Python command-line script, sender should generate a full python sender script that opens a TLS session, produce payment message MT103 or ISO 20022 message, and deliver to the receiver.

---

2) TLS Without Hostname/SNI

To establish a raw TLS client-to-server connection **WITHOUT** using a hostname/SNI**, you need a** certificate or public key fingerprint. **This is often called** certificate pinning **or** public key pinning**.**

You have two options for pinning:

A. Certificate SHA-256 fingerprint

- This is the SHA-256 of the **entire server certificate** (DER-encoded).
- Use this to validate the exact certificate during the TLS handshake.
- This fingerprint uniquely identifies the **entire leaf certificate** of the receiver server.
- You can use this value for **certificate pinning**, which allows your client to trust the server's certificate directly, without relying on a hostname/SNI.
- Any mismatch between the received certificate and this fingerprint will cause the TLS client to reject the connection.

B. Public Key SHA-256 fingerprint (SPKI pin)

- This is the SHA-256 of the **SubjectPublicKeyInfo** section of the certificate (DER-encoded).
- Recommended for host-to-host or S2S raw TLS connections without SNI.
- You can use this for **public key pinning**.
- This is slightly more flexible than cert pinning: if the server rotates its certificate but keeps the same key, the connection remains trusted.

TLS **does not require SNI** when connecting to a single service on a single IP.

C. Hostname/SNI not strictly required

- Since you have either the certificate or public key fingerprint pinned, your client can verify the server identity **without relying on SNI**.
- This is useful for raw TLS connections in host-to-host (H2H) or direct IP-to-IP financial messaging.

D. Practical usage in H2H/S2S context

- In a **raw TLS financial client**, you can skip the SNI entirely and establish a secure connection as long as you verify the received certificate or SPKI against these fingerprints.
- After the TLS handshake, any payload (ISO 20022 pacs.008, MT103, or other binary financial messages) can be transmitted securely inside the TLS stream.

- # Certificate SHA-256 fingerprint (leaf cert):

Which one should a sender use?

It depends on the **client software**, not on the fingerprint itself:

1. OpenSSL-style pinning (CLI, config files)

Usually expects **colon-separated uppercase**:

```
59:77:D1:92:7D:A4:C5:CC:58:67:DC:6B:44:C2:22:B9:FA:6A:D7:61:5D:DD:FE:40:BA:D3
:41:70:69:A0:F7:5D
```

2. Programmatic pinning (Java, Go, Python, Node, banking engines)

Almost always expects **raw hex lowercase**:

```
5977d1927da4c5cc5867dc6b44c222b9fa6ad7615dddfe40bad3417069a0f75d
```

So as sender:

Use whichever format your TLS layer expects, because both represent the same fingerprint.

If you're unsure, the safest universal choice is:

```
5977d1927da4c5cc5867dc6b44c222b9fa6ad7615dddfe40bad3417069a0f75d
```

- # Public Key SHA-256 fingerprint (SPKI pin):

```
68b3be528835e38b461c73b983dded53e089f22fdcb59cb3d9d730d17c612ae9
```

---

3) Strongly recommended (operational / security)

- **mTLS credentials** (client certificate + private key) if the receiver requires mutual TLS.
- **CA bundle** (if server uses a private CA).
- **Framing rules** (length-prefix, STX/ETX, UETR placement, timeouts)
- **ACK/NACK format** and behaviour (timers, retries, idempotency rules such as UETR or MsgId).
- **Source IP allowlist** (so receiver can whitelist your sending IP).
- **Logging / audit requirements** (what to record: MsgId, timestamps, UETR, TLS session fingerprint, response file).

---

4) Not required / usually irrelevant for the sender to open a TLS socket

- **Receiver Internal IP Range / Receiver DNS Range** (`172.16.0.0/24`, `192.168.1.100/24`) — these are internal network details for the receiver's routing or firewall; they do not matter for a sender that connects to the public IP, except when coordinating firewall rules.
- **Receiver server "friendly name"** (DEV-CORE-PAY-GW-01) — only useful for ops inventory, not for TLS itself.

---

5) Framing & payload notes (what the operator must confirm before sending)

TLS only gives you a secure pipe — **it does not define message boundaries**. You must confirm the receiver's framing rules. Common patterns:

- **Length-prefix framing**: 4-byte big-endian length followed by that many payload bytes (most common for bank H2H).
- **STX/ETX** framing: `0x02 ...data... 0x03` markers.
- **FIN prefix** for SWIFT-like frames (some environments used a FIN header).
- **Message idempotency**: UETR (UUID) or MsgId inside the message must be unique and handled as idempotency keys.

---

6) Summary (concise)

- **Essential for raw TLS S2S:** `IP`, `Port`, `SNI` and a reliable `certificate fingerprint / CA chain`.
- **Useful / required operationally:** `mTLS credentials`, `framing rules`, `ACK/NACK spec`, `UETR/idempotency`, and firewall/source IP allowlisting.
- **Internal/private ranges** are **not required** to make the TLS connection but matter for network/firewall design and receiver internal routing.
